

LARAVEL LOGS EXPOSER

Red Team Edition

by Dutchman Security

Idea by overthrash1337

Version 2.2 | For Authorized Testing Only | June 2026

Fingerprint, Locate, and Exfiltrate exposed Laravel artifacts.

Table of Contents

1. Introduction
2. Installation
3. Quick Start
4. Command-Line Flags
5. Usage Examples
 - 5.1 Single Target Scan
 - 5.2 Terminal Output Mode
 - 5.3 Store Results to Disk
 - 5.4 Custom Output Path
 - 5.5 Batch Mode
 - 5.6 Proxy Support
 - 5.7 Performance Tuning
6. Output & Report Format
7. Detection Methodology
8. Legal Disclaimer

1. Introduction

Laravel Logs Exposer is a professional red-team tool designed to identify and extract exposed log files, configuration dumps, and sensitive artifacts from Laravel-based web applications. It uses a multi-phase approach: first fingerprinting the application to confirm it runs Laravel, then systematically probing hundreds of known log-file and configuration paths, and finally analysing the results to prioritise the most critical findings.

The tool supports both single-target and batch operations, SOCKS/HTTP proxies for stealth, and exports comprehensive JSON reports suitable for inclusion in penetration-test deliverables.

2. Installation

Ensure Python 3.8+ is installed, then run:

```
pip install -r requirements.txt
```

The requirements file includes:

- requests & requests[socks] - HTTP client with SOCKS proxy support
- colorama - coloured terminal output
- urllib3 - underlying HTTP transport
- fpdf2 - PDF documentation generator (optional)

3. Quick Start

Point the tool at a Laravel application:

```
python3 logs-fucker.py -host https://target.com
```

By default findings are displayed in the terminal. Use -store to persist results or -o to save a single report.

4. Command-Line Flags

-host, --host URL

Single target URL (e.g. https://example.com)

-batch, --batch FILE

Path to a .txt file with one target URL per line

-terminal, --terminal

Display discovered paths live in the terminal

-store, --store [DIR]

Auto-save reports to ./<DIR>/<domain>/<ts>/report.json

-o, --output FILE

Save single-target report to a specific JSON file

-t, --threads N

Concurrent threads per target (default: 15)

--timeout N

Request timeout in seconds (default: 10)

--proxy URL

Proxy for all requests (http, socks4, socks5)

-v, --verbose

Show response content snippets

--no-color

Disable coloured terminal output

-h, --help

Show help message and exit

5. Usage Examples

5.1 Single Target Scan

```
python3 logs-fucker.py -host https://example.com
```

5.2 Terminal Output Mode

```
python3 logs-fucker.py -host https://example.com -terminal
```

5.3 Store Results to Disk

```
python3 logs-fucker.py -host https://example.com -store
```

Results are saved to `./output/example.com/<timestamp>/report.json`

5.4 Custom Output Path

```
python3 logs-fucker.py -host https://example.com -o ./reports/example.json
```

5.5 Batch Mode

```
python3 logs-fucker.py -batch targets.txt -store
```

Format of targets.txt (one URL per line):

```
https://target1.com
https://target2.com
# comments are ignored
https://target3.com
```

5.6 Proxy Support

```
python3 logs-fucker.py -host https://example.com --proxy socks5://127.0.0.1:9050
```

```
python3 logs-fucker.py -batch targets.txt -store --proxy http://proxy.corp:8080
```

Supported proxy schemes: http, https, socks4, socks5, socks5h.

5.7 Performance Tuning

```
python3 logs-fucker.py -host https://example.com -t 30 --timeout 5
```

Increase threads for faster scans; decrease timeout to avoid hanging on slow endpoints. The default 15 threads / 10 s timeout works for most targets.

6. Output & Report Format

Each scan produces a JSON report containing the complete A-to-Z findings:

tool / version

Tool name and version identifier

target

Scanned URL

scan_timestamp

UTC timestamp of the scan

is_laravel / laravel_confidence

Whether Laravel was detected and confidence score

laravel_indicators

List of fingerprint matches (cookie, header, path, body)

paths_scanned

Total number of paths probed

elapsed_seconds

Scan duration

summary

Counts by risk level: critical, high, medium, low, info

findings[]

Array of findings, each with:

Each finding entry includes: path, full URL, HTTP status, human-readable size, whether the content was classified as a log file, detected content-type, assigned risk level, and a snippet of the content (first 500 characters).

7. Detection Methodology

Phase 1 - Fingerprinting: The tool probes for 25+ Laravel indicators including cookies (laravel_session, laravel_token), HTTP headers (X-Powered-By, Set-Cookie), common Laravel endpoints (sanctum/csrf-cookie, _debugbar/open, telescope/health-check), and body-text patterns (csrf-token, livewire, stack traces). Each indicator scores points; a score of 4+ confirms Laravel.

Phase 2 - Path Discovery: 600+ known Laravel artifact paths are probed concurrently. These include storage log files (laravel.log, dated logs, JSON logs, compressed logs), environment files (.env, .env.backup), bootstrap cache, Telescope/Horizon/Debugbar endpoints, git leaks, and source-code exposures.

Log Detection: Content is classified as a genuine log file only when it contains strong indicators: stack-trace frames (#0 /path), Monolog channel prefixes (production.ERROR:), SQLSTATE errors, APP_KEY dumps, PHP array/object dumps, or exception chains.

Phase 3 - Risk Analysis: Findings are tagged CRITICAL (.env leaks, storage/logs with real log content, .git leaks), HIGH (Telescope, Horizon, Debugbar, bootstrap cache), MEDIUM (composer, artisan, routes), or LOW (other accessible files).

8. Legal Disclaimer

This tool is provided for authorised security assessments only. You must have explicit written permission from the target owner before using this tool. Unauthorised use may violate computer fraud and abuse laws in your jurisdiction. The authors and contributors assume no liability for misuse.

Developed by Dutchman Security

Idea by overthrash1337

For Authorized Testing Only | June 2026